

Premiers pas avec SQL

Création et mise à jour des tables

à retenir : les mots **CREATE** et **INSERT**

Dessiner le schéma de la base

```
Entrée[1]: -- création de la table eleve et classe avec des contraintes

DROP TABLE IF EXISTS eleve;
CREATE TABLE eleve (
    id INTEGER NOT NULL,
    nom TEXT NOT NULL,
    prenom TEXT,
    classe VARCHAR(2) NOT NULL,
    PRIMARY KEY(id) -- contrainte d'unicité de clé primaire
    FOREIGN KEY(classe) -- contrainte d'intégrité référentielle
    REFERENCES classe(idclasse));
    -- et doit être une des id de la table classe

DROP TABLE IF EXISTS classe;
CREATE TABLE classe (
    idclasse VARCHAR(2) NOT NULL, -- noms des classes de la table élève
    profppal TEXT NOT NULL,
    PRIMARY KEY(idclasse));
```

Clé, clé primaire, clé étrangère

Clé

Une *clé* d'une relation (table) est un ensemble d'un (ou plus) attribut·s qui suffit à déterminer les autres attributs.

Supposons que A est une clé d'une relation dont les attributs sont A, B, C, D . Si je connais la valeur de l'attribut A pour un n-uplet, je peux connaître les valeurs des 3 autres attributs.

Si A est une *clé* d'une relation dont les attributs sont A, B, C, D , on ne **peut pas** trouver ceci :

A	B	C	D
1	a	aa	aaa

2	b	bb	bbb
---	---	----	-----

1	x	xx	xxx
---	---	----	-----

Ici nous avons 2 occurrences de la même valeur de *A* suivies de valeurs différentes pour les autres attributs, ce qui fait que connaître la valeur de *A*, ici 1, ne me permet pas de connaître celles des autres attributs, puisque par exemple, les valeurs de l'attributs *B* pourraient être a ou x.

et si l'on trouvait :

A	B	C	D
1	a	aa	aaa
2	b	bb	bbb
1	a	aa	aaa

Ce serait tout simplement un doublon à éliminer.

clé primaire

Il peut exister plusieurs clés. On en choisit une comme *clé primaire*. Elle permet de distinguer d'éventuels doublons sur les autres attributs :

A	B	C	D
1	a	aa	aaa
2	b	bb	bbb
3	a	aa	aaa

EXEMPLE

Une table décrivant des personnes avec les attributs (colonnes) : *nom*, *prénom*, *dateNaissance*.

Le *nom* peut-il être une clé ?

Le *prénom* peut-il être une clé ?

La *dateNaissance* peut-elle être une clé ?

L'ensemble *nom*, *prénom* peut-il être une clé ?

La réponse à toutes ces questions est *non*, car il existe des personnes qui ont même nom, même prénom, mais des dates de naissances différentes. Ou la même date de naissance, mais pas le même nom. Etc.

La seule clé serait l'ensemble des attributs lui-même. Encore que deux personnes peuvent partager nom, prénom, et date de naissance.

La bonne pratique serait de donner à chaque personne un n° unique, donc d'ajouter un attribut *numero* à notre table. Cet attribut serait alors notre clé, et notre clé primaire.

Clé étrangère

La (les) *clé·s étrangère·s* sont des attributs d'une table qui sont des clés primaires d'autres tables. Ici, **les valeurs de l'attribut *classe* dans la table *eleve* existent dans la colonne *classe* de la table *classe***, comme indiqué par le mot `REFERENCES` dans mes instructions de définition de la table

Le fait d'inscrire ces éléments dans les instructions de définition des tables va empêcher l'utilisateur final d'inscrire des valeurs incorrectes.

```
Entrée[2]: /* remplissage de la table eleve */
```

```
INSERT INTO eleve
(id, nom, classe)
VALUES
(1, 'Tintin', 'C1')
```

Error: FOREIGN KEY constraint failed

Le SGBD nous gronde parce que les classes ne sont pas définies. Or, la valeur de l'attribut `classe` DOIT exister dans la table `classe`. C'est la **contrainte d'intégrité référencielle**

On remplit donc d'abord la table `classe`.

```
Entrée[3]: -- ATTENTION : À N'EXÉCUTER QU'UNE FOIS
```

```
INSERT INTO classe
VALUES
('C1', 'Hergé'),
('C2', 'Franquin');
```

maintenant, on peut remplir la table `eleve`.

```
Entrée[4]: INSERT INTO eleve
(id, nom, prenom, classe)
VALUES
(1, 'Tintin', '', 'C1'),
(2, 'Spirou', '', 'C2'),
(3, 'Haddock', 'Archibald', 'C1'),
(4, 'Castafiore', 'Bianca', 'C1'),
(5, 'Lagaffe', 'Gaston', 'C2'),
(9, 'Tsuno', 'Yoko', 'C5');
```

Error: FOREIGN KEY constraint failed

Le SGBD râle car C5 n'est pas dans la table classe.

```
Entrée[5]: -- on retire tout !
DELETE FROM eleve
where 1;
-- on remet tout
INSERT INTO eleve
(id, nom, prenom, classe)
VALUES
(1, 'Tintin', '', 'C1'),
(2, 'Spirou', '', 'C2'),
(3, 'Haddock', 'Archibald', 'C1'),
(4, 'Castafiore', 'Bianca', 'C1'),
(5, 'Lagaffe', 'Gaston', 'C2'),
(9, 'Tsuno', 'Yoko', 'C2');
```

Mise à jour d'une table

On utilise les mots **UPDATE** et **DELETE**

```
Entrée[6]: -- Spirou part en classe C1
-- À n'exécuter qu'une fois
UPDATE eleve
SET classe = "C1"
WHERE nom = "Spirou";
-- Gaston Lagaffe s'est fait virer
DELETE FROM eleve
WHERE id = 5; -- attention aux doublons éventuels
```

Interrogation d'une table

```
Entrée[7]: -- allez on reprend Gaston
-- une seule fois
INSERT INTO eleve
VALUES
(5, 'Lagaffe', 'Gaston', 'C2');
```

```
Entrée[8]: -- résultat
SELECT *
FROM eleve;
```

Sortie[8]:

id	nom	prenom	classe
1	Tintin		C1
2	Spirou		C1
3	Haddock	Archibald	C1
4	Castafiore	Bianca	C1
5	Lagaffe	Gaston	C2
9	Tsuno	Yoko	C2

Interrogation

À partir de là, on peut exécuter plusieurs fois une même cellule du notebook puisque les requêtes d'interrogation ne modifient pas les tables.

```
SELECT * FROM <nom_table>
```

va nous donner toutes les lignes et toutes les colonnes de la table.

projection

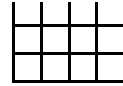
```
SELECT <nom_attribut>, <nom_attribut> FROM <nom_table>
```

sélectionne une ou plusieurs colonnes (correspondant aux attribut indiqués) :

restriction :

```
SELECT * FROM <nom_table> WHERE <condition>
```

sélectionne parmi les lignes celles qui valident une certaine condition



les conditions courantes :

`<nom_attribut> [=, >, <, <=, >=] <valeur>`

`<nom_attribut> LIKE <str avec caractères génériques comme % ou _>`

Le caractère `%` représente n'importe quelle chaîne de caractères. On peut l'insérer n'importe où dans la chaîne de caractères de comparaison.

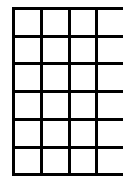
Par ex :

```
SELECT *
FROM eleve
WHERE nom LIKE '%a%'
```

sélectionne toutes les lignes où la valeur de l'attribut *nom* est une chaîne de caractères qui contient un *a*.

restriction avec projection :

On va combiner les deux pour obtenir seulement les cases rouges



Exemple : quels sont les prénoms des élèves de la classe C2 ?

```
Entrée[9]: SELECT prenom
FROM eleve
WHERE classe = 'C2'
```

Sortie[9]:

prenom

Gaston

Yoko

Fonctions d'agrégation

Une fois le résultat d'une requête SELECT obtenu, on peut lui appliquer une fonction d'agrégation, appelée ainsi parce qu'elle va retourner un seul résultat à partir d'une pluralité. Par ex, on veut savoir le nombre de prénoms de la classe C2.

Dans une telle requête, il faut bien décomposer les choses pour construire la requête : d'abord, le résultat qui va faire un certain nombre de lignes, puis la fonction d'aggrégat qui est appliquée sur ce résultat.

Par ex. combien d'élèves dans la classe C1 ?

```
Entrée[10]: /* pas bon !*/  
SELECT count(prenom)  
FROM eleve  
WHERE classe = 'C1'
```

```
Sortie[10]:
```

<u>count(prenom)</u>
4

On risque que `count` ne compte que les valeurs non nulles et certains élèves n'ont pas de prénom. Il vaut mieux compter le nombre d'id ou le nombre de lignes complètes, tout simplement.

```
Entrée[11]: SELECT count(*)  
FROM eleve  
WHERE classe = 'C1'
```

```
Sortie[11]:
```

<u>count(*)</u>
4

`count` compte bien le nombre de lignes non nulles. Par ex, si on veut savoir le nombre de classes dans la table *e/leve*

```
Entrée[12]: /* pas bon */  
SELECT count(classe)  
FROM eleve
```

```
Sortie[12]:
```

<u>count(classe)</u>
6

patatras ! Il n'y a pas 6 classes différentes. Mais quand on fait une projection sur la colonne *classe*, on récupère bien 6 lignes, **dont plusieurs identiques**.

Règle d'or : on comprend le résultat d'une requête AVANT d'appliquer une fonction sur ce résultat.

```
Entrée[13]: SELECT classe  
FROM eleve;
```

```
Sortie[13]:
```

<u>classe</u>
C1
C1
C1
C1
C2
C2

Il faut utiliser le mot clé **DISTINCT**.

```
Entrée[14]: SELECT count(DISTINCT classe) as "nombre de classes"  
FROM eleve  
-- as sert juste à renommer un attribut du select ici
```

```
Sortie[14]:
```

<u>nombre de classes</u>
2

Les fonctions **MAX, MIN, AVG, SUM** ont un fonctionnement similaire.

Tri des résultats

Ajouter **ORDER BY** pour trier les résultats dans l'ordre croissant des valeurs de *nom_colonne*. Ajouter **DESC** pour un ordre décroissant.

```
Entrée[15]: SELECT nom  
            FROM eleve  
            ORDER BY nom DESC
```

Sortie[15]:

nom
Tsuno
Tintin
Spirou
Lagaffe
Haddock
Castafiore

Jointures

Il est fréquent que l'information soit dispersée dans deux tables.

Dans ce cas, les lignes qui constituent le résultat de notre requête seront composées de lignes en provenance de différentes tables (celles dont on a besoin).

Par ex. **quel est le prof principal de Tintin ?**

Nous avons besoin d'une ligne de la table `élève`, à cause du prénom, et d'une ligne de la table `classe`, pour obtenir le nom du prof principal.

Nous mettrons donc ces deux tables dans le `FROM`

```
FROM eleve, table
```

Quand l'information est dispersée dans deux tables T1 et T2, la jointure consiste à construire d'abord une sorte de grosse table où chaque ligne est composée d'une ligne de T1 suivie d'une ligne de T2 (on appelle cette étape le produit cartésien de T1 avec T2). Cette grosse table contient le nombre de lignes dans T1 multiplié par le nombre de lignes dans T2.

Entrée[16]: **SELECT ***
FROM eleve, classe;

Sortie[16]:

id	nom	prenom	classe	idclasse	profppal
1	Tintin		C1	C1	Hergé
1	Tintin		C1	C2	Franquin
2	Spirou		C1	C1	Hergé
2	Spirou		C1	C2	Franquin
3	Haddock	Archibald	C1	C1	Hergé
3	Haddock	Archibald	C1	C2	Franquin
4	Castafiore	Bianca	C1	C1	Hergé
4	Castafiore	Bianca	C1	C2	Franquin
5	Lagaffe	Gaston	C2	C1	Hergé
5	Lagaffe	Gaston	C2	C2	Franquin
9	Tsuno	Yoko	C2	C1	Hergé
9	Tsuno	Yoko	C2	C2	Franquin

Le produit cartésien contient encore beaucoup trop d'informations et une telle jointure n'a pas grand intérêt.

Attention, si vous faites des décomptes (SUM, COUNT, etc) avec une jointure de ce type, vous risquez d'avoir des résultats surprenants car il y aura beaucoup trop de lignes !

En face de la ligne d'un élève, on veut seulement la ligne de la table `classe` qui correspond à la classe de cet élève

-> repérez les lignes qui nous intéressent dans la sortie de la cellule précédente

On va prendre seulement les lignes dans lesquelles **l'attribut** `classe` dans la table `élève` a la même valeur que l'id de classe dans la table `classe`.

Entrée[17]:

```
SELECT *
FROM eleve, classe
WHERE eleve.classe = classe.idclasse
-- restriction aux lignes intéressantes
```

Sortie[17]:

id	nom	prenom	classe	idclasse	profppal
1	Tintin		C1	C1	Hergé
2	Spirou		C1	C1	Hergé
3	Haddock	Archibald	C1	C1	Hergé
4	Castafiore	Bianca	C1	C1	Hergé
5	Lagaffe	Gaston	C2	C2	Franquin
9	Tsuno	Yoko	C2	C2	Franquin

Vous remarquez la sélection de lignes. Nous avons, pour chaque élève, sa classe donc son prof principal.

Cette table est celle que nous aurions fabriquée si nous avions voulu mettre toutes les infos dans une seule table. L'information sur les classes est répétée inutilement.

Il est possible d'obtenir le même résultat en utilisant les clauses `JOIN ... ON`.

On complète par une restriction pour obtenir seulement l'information sur Tintin.

Entrée[18]:

```
SELECT *
FROM eleve
JOIN classe
ON eleve.classe = classe.idclasse
WHERE nom = 'Tintin'
```

Sortie[18]:

id	nom	prenom	classe	idclasse	profppal
1	Tintin		C1	C1	Hergé

Et on finit par une projection (`SELECT ...`) pour avoir seulement le nom du prof

Entrée[19]:

```
SELECT profppal as "Professeur principal"
FROM eleve
JOIN classe
ON eleve.classe = classe.idclasse
WHERE nom = 'Tintin'
```

Sortie[19]:

Professeur principal
Hergé

Il arrive que les colonnes similaires dans les deux tables aient le même nom. Ici, c'est inutile, mais on aurait pu faire également :

```
Entrée[20]: SELECT profppal
FROM eleve
JOIN
classe
ON eleve.classe = classe.idclasse
WHERE nom = 'Tintin';
```

Sortie[20]:

profppal
Hergé

Il est possible de faire des jointures entre 2, 3... autant de tables que l'on veut lorsque l'information intéressante est dispersée entre toutes ces tables.

Il est également possible de faire le produit cartésien d'une table avec elle-même, sous réserve de renommer les attributs (car deux colonnes ne doivent pas avoir le même nom).

```
Entrée[21]: select * from eleve as e1 join eleve as e2
```

Sortie[21]:

	id	nom	prenom	classe	id	nom	prenom	classe
	1	Tintin		C1	1	Tintin		C1
	1	Tintin		C1	2	Spirou		C1
	1	Tintin		C1	3	Haddock	Archibald	C1
	1	Tintin		C1	4	Castafiore	Bianca	C1
	1	Tintin		C1	5	Lagaffe	Gaston	C2
	1	Tintin		C1	9	Tsuno	Yoko	C2
	2	Spirou		C1	1	Tintin		C1
	2	Spirou		C1	2	Spirou		C1
	2	Spirou		C1	3	Haddock	Archibald	C1
	2	Spirou		C1	4	Castafiore	Bianca	C1
	2	Spirou		C1	5	Lagaffe	Gaston	C2
	2	Spirou		C1	9	Tsuno	Yoko	C2
	3	Haddock	Archibald	C1	1	Tintin		C1
	3	Haddock	Archibald	C1	2	Spirou		C1
	3	Haddock	Archibald	C1	3	Haddock	Archibald	C1

3	Haddock	Archibald	C1	4	Castafiore	Bianca	C1
3	Haddock	Archibald	C1	5	Lagaffe	Gaston	C2
3	Haddock	Archibald	C1	9	Tsuno	Yoko	C2
4	Castafiore	Bianca	C1	1	Tintin		C1
4	Castafiore	Bianca	C1	2	Spirou		C1
4	Castafiore	Bianca	C1	3	Haddock	Archibald	C1
4	Castafiore	Bianca	C1	4	Castafiore	Bianca	C1
4	Castafiore	Bianca	C1	5	Lagaffe	Gaston	C2
4	Castafiore	Bianca	C1	9	Tsuno	Yoko	C2
5	Lagaffe	Gaston	C2	1	Tintin		C1
5	Lagaffe	Gaston	C2	2	Spirou		C1
5	Lagaffe	Gaston	C2	3	Haddock	Archibald	C1
5	Lagaffe	Gaston	C2	4	Castafiore	Bianca	C1
5	Lagaffe	Gaston	C2	5	Lagaffe	Gaston	C2
5	Lagaffe	Gaston	C2	9	Tsuno	Yoko	C2
9	Tsuno	Yoko	C2	1	Tintin		C1
9	Tsuno	Yoko	C2	2	Spirou		C1
9	Tsuno	Yoko	C2	3	Haddock	Archibald	C1
9	Tsuno	Yoko	C2	4	Castafiore	Bianca	C1
9	Tsuno	Yoko	C2	5	Lagaffe	Gaston	C2
9	Tsuno	Yoko	C2	9	Tsuno	Yoko	C2

Quel intérêt ? eh bien, par exemple si l'on veut deux élèves qui ont le même prénom.

```
Entrée[22]: delete from eleve where id = 10;
insert into eleve values (10, "Rebuffat", "Gaston", "C2")
```

```
Entrée[23]: select *  
from eleve as e1, eleve as e2  
where e1.prenom = e2.prenom  
and e1.nom != e2.nom
```

```
Sortie[23]:
```

id	nom	prenom	classe	id	nom	prenom	classe
1	Tintin		C1	2	Spirou		C1
2	Spirou		C1	1	Tintin		C1
5	Lagaffe	Gaston	C2	10	Rebuffat	Gaston	C2
10	Rebuffat	Gaston	C2	5	Lagaffe	Gaston	C2

Que remarquez-vous ?

Les dates

La gestion des dates est parfois compliquée en SQL

https://www.sqlite.org/lang_datefunc.html (https://www.sqlite.org/lang_datefunc.html)

Tous ou presque utilisent le format ISO 8601 :

`YYYY-MM-JJ hh:mm:ss`

Par ex, le 1 juillet 2018 à 15 heures 33 s'écrit

`'2018-07-01 15:33:00'`

L'avantage, c'est que si la date A représente un moment arrivé *après* à la date B, A sera également classé *après* B dans l'ordre lexicographique.

Pour sqlite, utiliser la fonction

`strftime(<modifieur>, <date>)`

Le modifieur étant à choisir parmi cette liste :

<code>%d</code>	day of month: 00
<code>%f</code>	fractional seconds: SS.SSS
<code>%H</code>	hour: 00-24
<code>%j</code>	day of year: 001-366
<code>%J</code>	Julian day number
<code>%m</code>	month: 01-12
<code>%M</code>	minute: 00-59
<code>%s</code>	seconds since 1970-01-01
<code>%S</code>	seconds: 00-59
<code>%w</code>	day of week 0-6 with Sunday==0
<code>%W</code>	week of year: 00-53
<code>%Y</code>	year: 0000-9999

Pour calculer des durées, il faut faire des soustractions entre résultats des fonctions `strftime` et/ou `julianday`.

```
Entrée[24]: -- on extrait l'année d'une date
SELECT '2018-10-04 12:43:00', strftime('%Y','2018-10-04 12:43:00')
```

```
Sortie[24]:          '2018-10-04 12:43:00'  strftime('%Y','2018-10-04 12:43:00')
          -----
          2018-10-04 12:43:00                                2018
```

Entrée[25]: *-- on extrait le mois d'une date*
SELECT strftime('%m','2018-10-04 12:43:00')

Sortie[25]: strftime('%m','2018-10-04 12:43:00')
 10

Entrée[26]: *-- le nombre (fractionnaire) de jours depuis le début du calendrier J*
SELECT julianday('2018-10-04 12:43:00')

Sortie[26]: julianday('2018-10-04 12:43:00')
 2458396.029861111

Entrée[27]: *-- le nombre de secondes depuis 1970-01-01*
SELECT strftime('%s', '1970-01-01')

Sortie[27]: strftime('%s', '1970-01-01')
 0

Entrée[28]: **SELECT** strftime('%s', '2021-11-13')
-- 86400 = le nombre de secondes dans un jour

Sortie[28]: strftime('%s', '2021-11-13') -- 86400 = le nombre de secondes dans un jour
 1636761600

Entrée[29]: **SELECT** julianday('2021-11-13') - julianday('1970-01-01')
-- on obtient bien le même nombre de jours

Sortie[29]: julianday('2021-11-13') - julianday('1970-01-01') -- on obtient bien le même nombre de jours
 18944

Quelques mots sur le GROUP BY

Imaginons qu'on veuille le nombre d'élèves par classe.